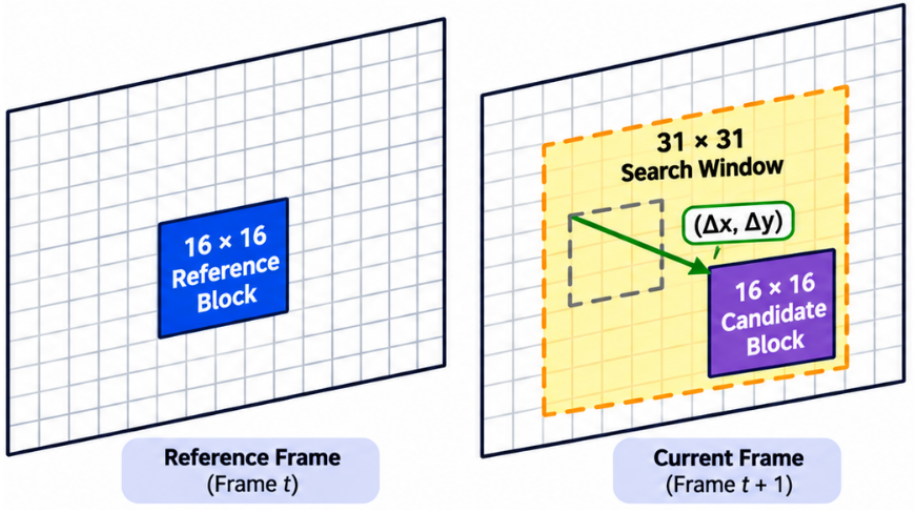
Advanced Digital Design
Motion Estimator

By: Jennifer Elias, Evan Caplinger, Huy Nguyen

Design Specifications

- 16x16 Reference Block
- 31x31 Search Window
- Grey-scale coded pixels
- Clocks available : 520 MHz
- 14 nm CMOS library



Design Objective:

The design is to read reference and search pixels from the testbench memory arrays (R_Ram and S_Ram) and process them using 16 Processing Element (PE) Modules that calculate and accumulate absolute differences.

The comparator within the design then selects the smallest Sum of Absolute Differences (SAD) and its corresponding motion coordinates.

- R_Ram Storing the 16x16 Reference Block Pixels
- S_Ram Storing the 31x31 Search Window Pixels

Simulation Modifications

- We chose to run the design at the required 520 MHz
- So, we modified timescale:
  - Original: timescale 1ns/10ps
  - Ours: timescale 1ns/100fs
- Clock period:
  - Original: always #10 clock = ~clock;
  - Ours: always #0.9615 clock = ~clock;
- All delays in testbench:
  - Original: @(posedge clock);#10;
  - Ours: @(posedge clock);#0.02;

Top-Level RTL Verification

VCS + Verdi

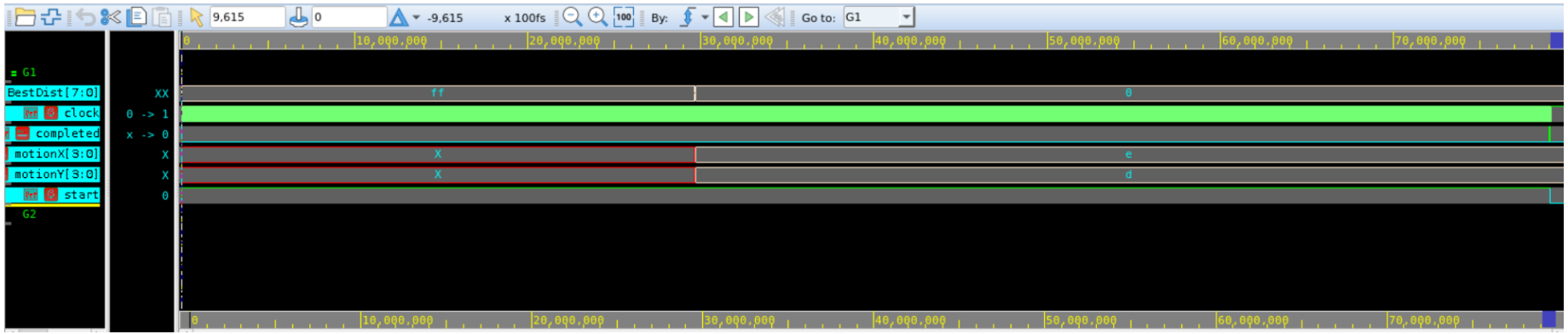


Figure 1

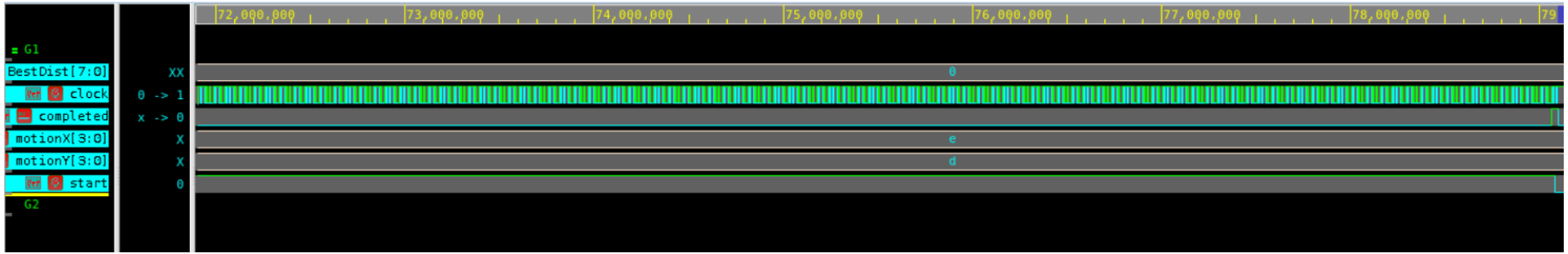


Figure 2

Top-Level RTL Verification (cont.)

VCS + Verdi

Using a 520 MHz clock, the Verdi waveform shows the minimum SAD (BestDist) reaching 0 and the resulting motion vector of (-2, -3), matching the expected displacement that was simulated earlier. The completed signal in Figure 2 indicates the end of processing at 7900 ns.

Signal	Observed Value	Meaning
BestDist[7:0]	00	Minimum SAD = 0; perfect match
motionX[3:0]	E	-2 in signed 4-bit representation
motionY[3:0]	D	-3 in signed 4-bit representation
start	HIGH during processing	Enables the motion estimation operation
completed	LOW → HIGH	Indicates processing is complete at approximately 7,900 ns

Waveform Interpretation

Testbench Design

VCS + Verdi

Translates the reference frame by Expected_motionX in the X-axis, Expected_motionY in the Y-axis:

```
    Expected_motionX = $urandom_range(0,15)-8;

    Expected_motionY = $urandom_range(0,15)-8;



    //extract Rmem from Smem for Expected_motionX and Expected_motionY



    foreach (Rmem[i]) begin

            Rmem[i] = Smem[32*8+8+(((i/16)+Expected_motionY)*32)+
((i%16)+Expected_motionX)];

        end
```

Controller RTL Verification

VCS + Verdi

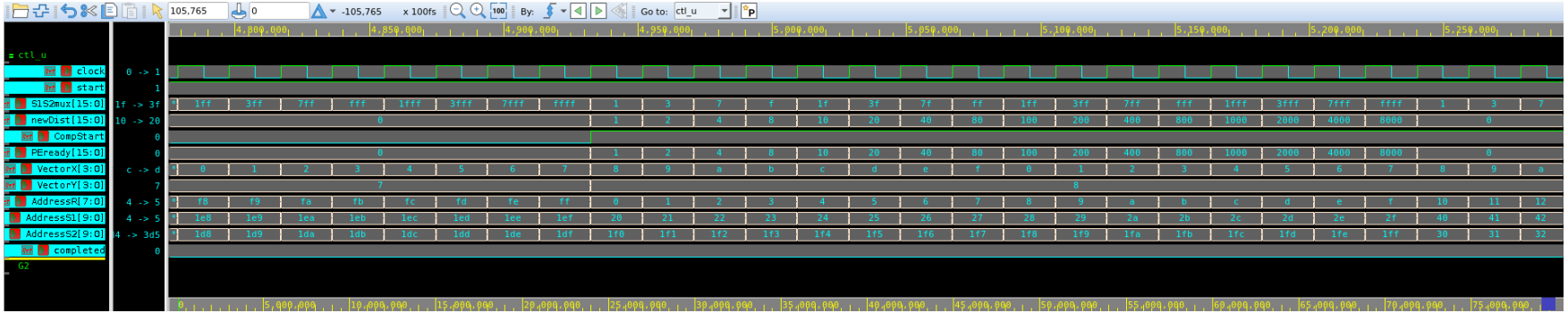


Figure 3: First row transition

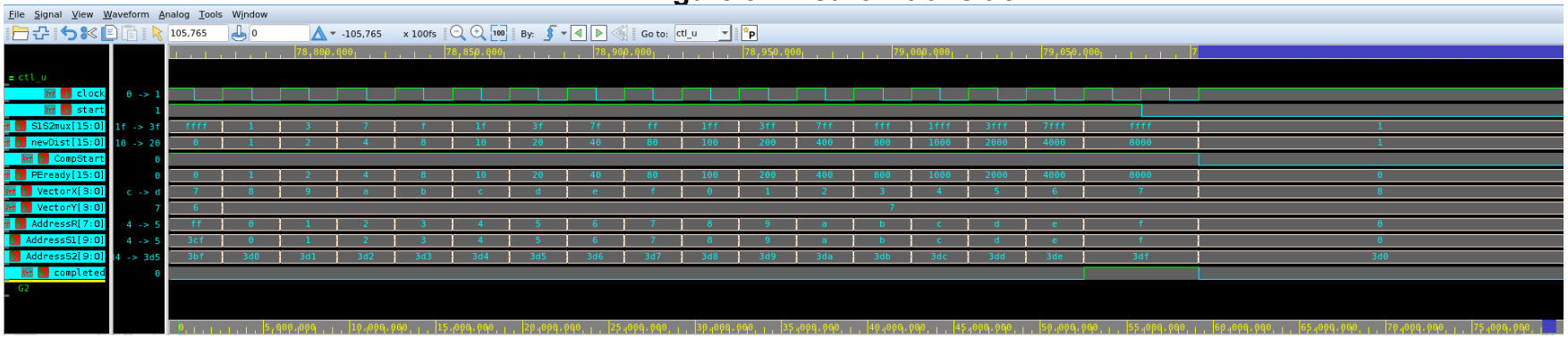


Figure 4: Module completion

Controller RTL Verification (cont.)

VCS + Verdi

Signal	Observed Behavior	Meaning
S1S2mux	1 bit flipped every clock	At first clock, PE 0 chooses S2 over S1; then PE 1; etc.
newDist	Left shift every clock, first 16 clocks of row; 0 after until next row	At beginning of row, reinitialize PE 0 accumulator, then PE 1, PE 2, etc. each clock cycle
CompStart	High after first row read	Comparator should start after first row is accumulated
PEready	Same as newDist, but 0 for first row	Bitmap to determine which PE to compare; after each row, compare PE 0, then PE 1, etc.
VectorX	Counts up every tick	Determines X-value of new motion vector if new best found
VectorY	Counts up once per row	Determines Y-value of new motion vector if new best found; only increases when row increases

Controller RTL Verification (cont.)

VCS + Verdi

Signal	Observed Behavior	Meaning
AddressR	Count up to 8'hff; reset	Cycles through 256 bytes of ref image register to load
AddressS1	Read 16 bytes, skip 16; do this 10'h0 -> 10'h1ef; reset to 10'h20 -> 10'h20f; etc.	Reads in values; skips 16 bytes to achieve 2-way hardware parallelism with AddressS1 memory line
AddressS2	Same as AddressS2, but reads the 16 bytes AddressS1 doesn't read	Achieves 2-way parallelism with AddressS1
completed	Low until completion, then goes high	Interface bit that flags module completion

PE RTL Verification

VCS + Verdi

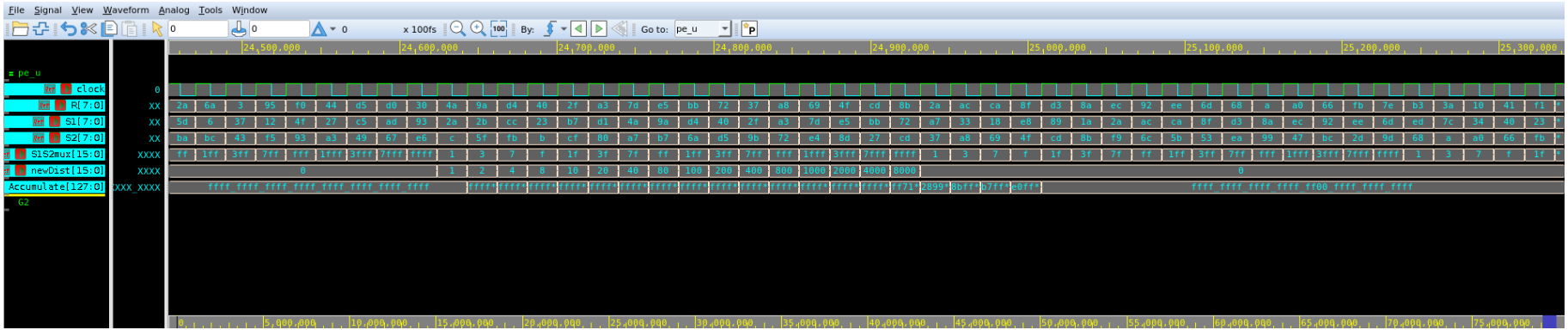


Figure 5: Motion vector found

PE RTL Verification (cont.)

VCS + Verdi

Signal	Observed Behavior	Meaning
R	Randomized image data	Randomly initialized reference data
S1	Randomized image data	Randomly initialized reference data; should be able to observe repetitions of 16-byte sequences from R by data construction
S2	Randomized image data	Same behavior as S1
S1S2mux, newDist	Behavior explained in control	Same interpretation as in controller verification
Accumulate	Values populate, then data appears in vector as 0's in otherwise fully saturated vector: ffff_ffff_ffff_ffff_ff00_ffff_ff	After new row is started, values for some accumulator slots dip as the accumulator for the associated PE resets. For the expected motion vector, PE never accumulates anything, as image is an exact copy.

Comparator RTL Verification

VCS + Verdi

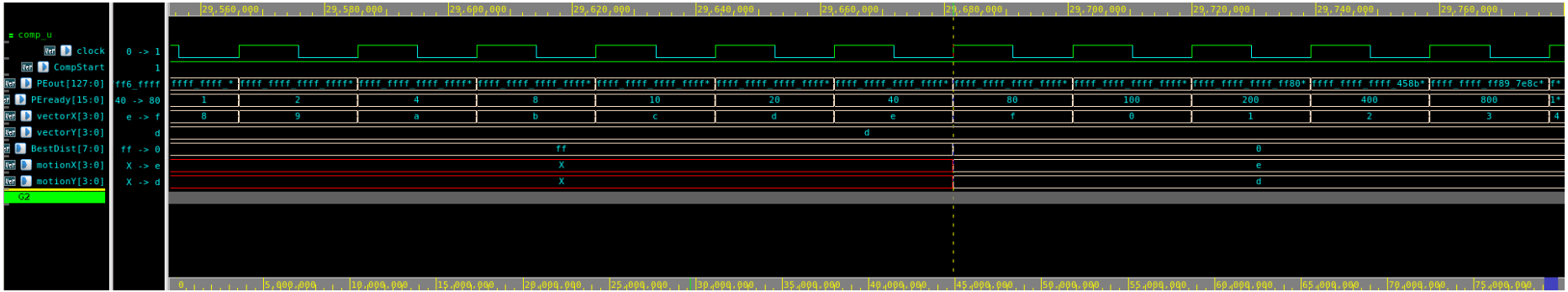


Figure 6: Motion vector found in comparator

Comparator RTL Verification (cont.)

VCS + Verdi

Signal	Observed Behavior	Meaning
CompStart, PEready, vectorX, vectorY	Same as in control module	Same as in control module
BestDist	Saturated until exact match as found, then goes to 0	When it goes to 0, that indicates an exact match. Edge of transition to 0 coincides with transitions of motionX, motionY
motionX, motionY	When BestDist decreases, takes last-clock vals of vectorX, vectorY	Gets set to the value of whatever vectorX and vectorY were when the new lowest distance (BestDist) was found

Direct Fixture & Perturbed Match Verification

top_tb_direct.v & top_tb.v | 520 MHz RTL Simulation

To complement randomized perfect-match testing (BestDist = 00), the directed fixture (top_tb_direct.v with ref.txt and search.txt), perturbed non-zero SAD stimulus (Rmem[0] += 3 at (-2, -3)), and upper-corner boundary (+7, +7) were simulated at 520 MHz (1ns/100fs, #0.9615).

```
// Exact cycle when PE[col_x] asserts PEready for candidate (Expected_motionX, Expected_motionY):
count_match = (Expected_motionY + 9) * 256 + (Expected_motionX + 8); // e.g. (-2,-3) -> 6*256 + 6 = 1542
```

Test Case / Signal	Observed Behavior (520 MHz)	Verification Meaning
Direct: PE0..PE14 (count 256..270)	newDist = 8'hFF (Carry = 1)	ref.txt has Rmem[0]=8'hFF; search.txt has Smem[0]=8'hFE, Smem[15]=8'hFF. PE0 sums |FF-FE|+|00-FF| = 256 -> saturates to 8'hFF
Direct: PE15 / PEend (count 271)	BestDist = 00, motionX = 7, motionY = 8	At t = 522.11 ns (count 271), PE15 aligns R[0,0]=8'hFF with S[0,15]=8'hFF (newDist=00), latching (+7, -8) on count = 272
Perturbed (-2, -3) (count 1542)	BestDist = 03, motionX = E, motionY = D	Perturbing Rmem[0] by +3 produces non-zero SAD = 3 in PE6 at count = 1542 (t = 2,965.27 ns), recovering (-2, -3)
Corner (+7, +7) (count 4111)	BestDist = 00, motionX = 7, motionY = 7	Final candidate (16*256 + 15 = 4111) asserts PEready[15], newBest=1, and completed=1 at t = 7,905.45 ns
Tie-Break (newDist < BestDist)	Strict < comparison in Comparator	BestDist initializes to 8'hFF when CompStart=0; strict inequality preserves earliest row-major winner on equal SAD

Pipeline Schedule & Coordinate Timing Analysis

control + PEtotal + Comparator | 520 MHz Datapath Timing

The controller (ctl_u) orchestrates all 256 candidate block comparisons using a single 13-bit counter count[12:0] (0 to 4111) without an FSM. Reference bytes shift across 15 Rpipe registers while dual search ports (S1, S2) keep all 16 PEs busy after a 15-cycle fill.

```
AddressR = count[7:0];  AddressS1 = (count[11:8] + count[7:4])*32 + count[3:0];  temp = count[11:0] - 16;
AddressS2 = (temp[11:8] + temp[7:4])*32 + temp[3:0] + 16;  VectorX = count[3:0] - 8;  VectorY = count[11:8] - 9;
```

Pipeline Signal / Stage	RTL Equation / Cycle Schedule	Architectural Meaning in top.v
Rpipe0..Rpipe14 & PEend	15-stage shift chain (count 0..14 fill)	PE0..PE14 register R into Rpipe on each posedge clock; PE15 (PEend) omits Rpipe since it is the final stage
newDist[15:0] & PEready[15:0]	newDist[i] = (count[7:0] == i)	PE[i] starts a new 256-pixel SAD at count[7:0]==i; PEready[i] asserts 256 cycles later when !(count < 256)
S1S2mux[15:0] & AddressS1/S2	S1S2mux[i] = (count[3:0] >= i)	Selects S1 when pixel column count[3:0] >= PE index i, else S2 (delayed by 16 cycles + 16 cols) for 2-way parallelism
VectorX[3:0], VectorY[3:0]	VectorX = count[3:0]-8, VectorY = count[11:8]-9	Because row_y finishes at count[11:8] = row_y + 1 (1..16), subtracting 9 maps completed rows to signed -8..+7
count_complete = 4111	4,111 cycles × 1.923 ns = 7,905.45 ns	16 rows × 256 cycles + 15 pipeline fill cycles = 4,111 cycles (~7,900 ns in Verdi; 126,489 block searches/s = 30.88 fps)

Synthesis Hierarchy, Memory Boundary & 520 MHz SDC

top.v Gate-Level Synthesis & 520 MHz Timing Constraints

Per ENGR 852 specifications, ROM_R (256×8) and ROM_S (1024×8) are instantiated only in the testbench (0 memory bits in module top). Synthesizing top.v yields 3,254 cells and 277 flip-flops constrained at 520 MHz (1.923 ns), 0.5 ns I/O delay, and 0.2 ns uncertainty.

```
create_clock -name clock -period 1.923 [get_ports clock];  set_clock_uncertainty 0.2 [get_clocks clock]
set_input_delay 0.5 -clock clock [remove_from_collection [all_inputs] [get_ports clock]];  set_output_delay 0.5 -clock
clock [all_outputs]
```

Module / Timing Parameter	Synthesized Count / Spec	Hardware Breakdown & Timing Budget Impact
PEtotal (15× PE + 1× PEend)	2,744 cells (84.3%), 248 DFFs	15× PE (172 cells, 16 DFFs: Accumulate[7:0] + Rpipe[7:0]) + 1× PEend (164 cells, 8 DFFs: Accumulate[7:0] only)
Comparator (comp_u)	353 cells (10.8%), 16 DFFs	16-to-1 PEout[127:0] mux, 8-bit comparator (newDist < BestDist), BestDist[7:0] (init 8'hFF) & motionX/Y[3:0] regs
control (ctl_u) & Top Boundary	157 cells (4.8%), 13 DFFs	13-bit count[12:0] & address logic; 12 top ports (71 bits: 26-bit inputs, 45-bit outputs, 0 synthesized RAM bits)
Internal Reg-to-Reg Budget	1.923 ns - 0.200 ns = 1.723 ns	520 MHz period minus 0.2 ns uncertainty; critical path in PE: S1S2mux -> |R - selected_s| -> 8-bit carry adder -> newDist mux
External I/O Path Budget	1.923 - 0.500 - 0.200 = 1.223 ns	0.5 ns input delay (R, S1, S2, start) & 0.5 ns output delay (AddressR, AddressS1/S2, BestDist, motionX/Y, completed)

Original RTL: 8-bit SAD Saturation

VCS + Verdi | Directed high-contrast test

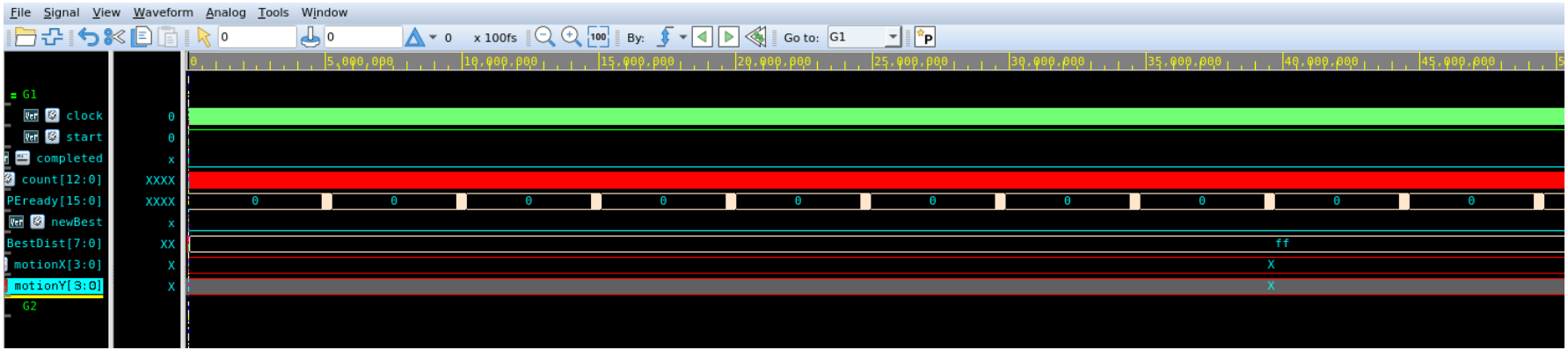
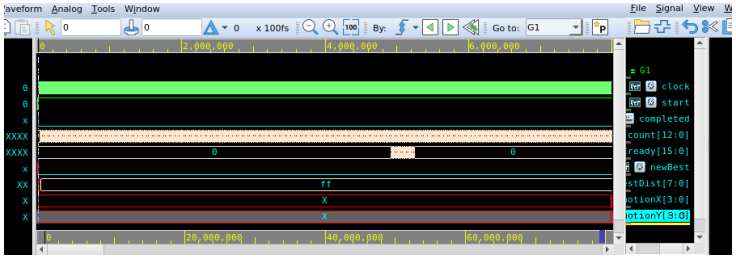


Figure 1. PEready updates while newBest stays low; BestDist remains FF.



At completion:
BestDist = FF
motionX = X, motionY = X

All 256 pixels differ by FF, so the correct SAD is FF00. The 8-bit comparator cannot select an FF candidate against an FF reset value.

16-bit SAD: Corrected Motion Vector

VCS + Verdi | Separate RTL and self-checking testbench

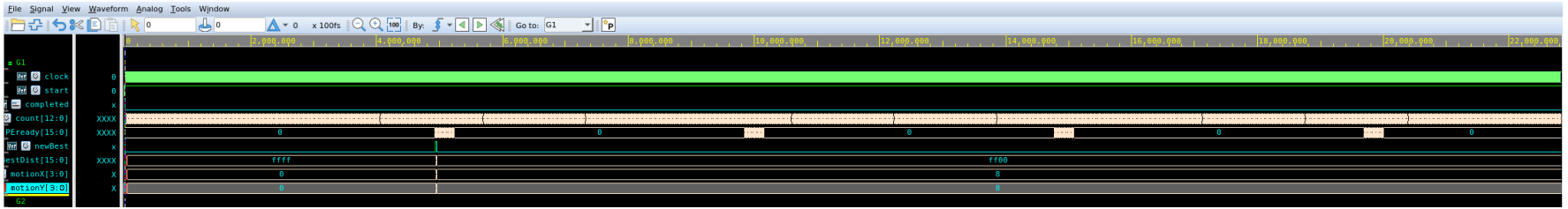


Figure 2. newBest asserts and BestDist updates from FFFF to FF00.

Original RTL

BestDist = FF
motionX / motionY = X / X

16-bit corrected RTL

BestDist = FF00
motionX / motionY = 8 / 8

Encoded 4-bit vector (8,8) is signed (-8,-8). With equal SADs, the first candidate wins; the completion-hold test kept outputs stable.

14 nm Synthesis: Area and Timing

Fusion Compiler | SAED14 RVT | 520 MHz

Mapped design comparison

Metric	Original 8-bit	16-bit V3B
Standard-cell area	881.96 µm²	1,193.03 µm²
Mapped cells	1,693	1,905
Sequential cells	280	416
Memory macros	0	0

Timing constraints

Clock: 1.923 ns (520 MHz)
Input delay: 0.5 ns
Output delay: 0.5 ns
Clock uncertainty: 0.2 ns

Compile-stage setup slack

+0.06 ns (MET)

The 16-bit datapath uses 311.07 µm² more cell area (+35.3%) to preserve the full SAD. Synthesis, floorplan and placement are recorded; final routed setup/hold are separate checks.

Reference and search memories are in the testbench, not in the synthesized motion estimator.